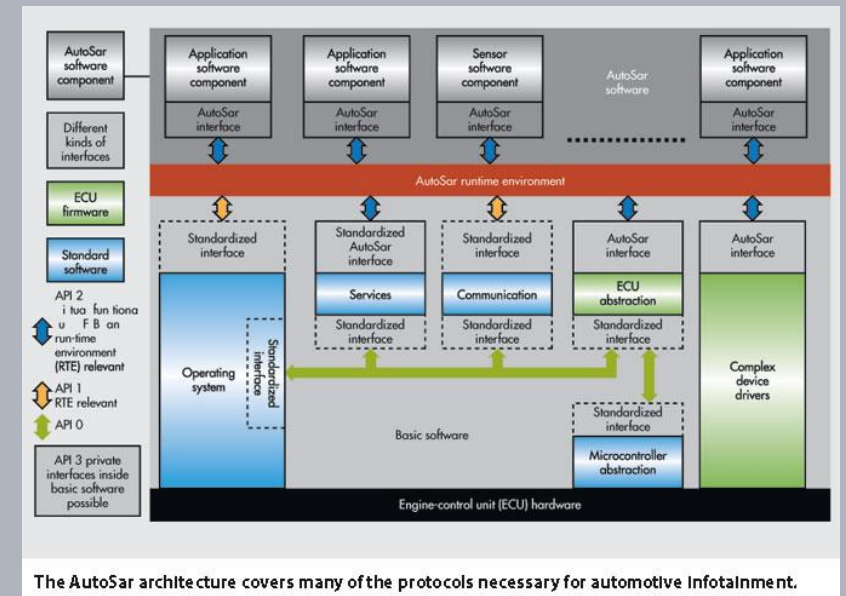


# Advanced Microcontroller Architectures and Embedded OS

## Embedded OS, Multitasking

Prof. Dr.-Ing. Peter Fromm

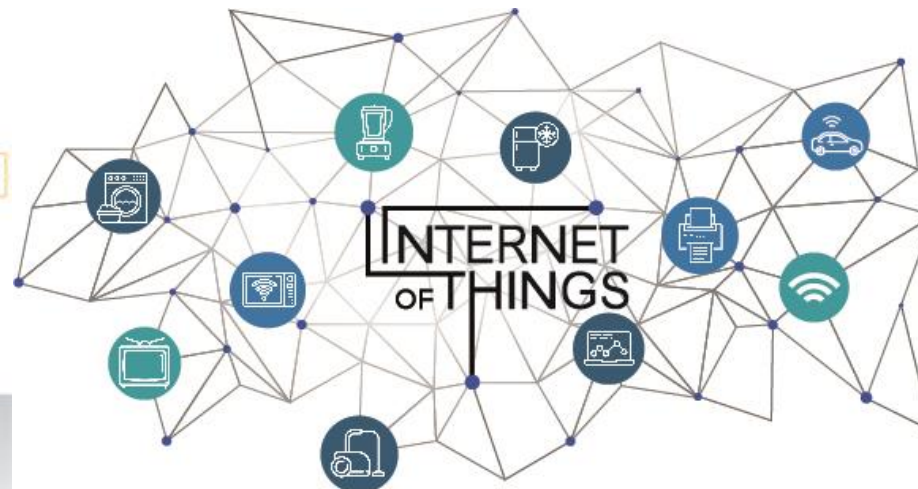
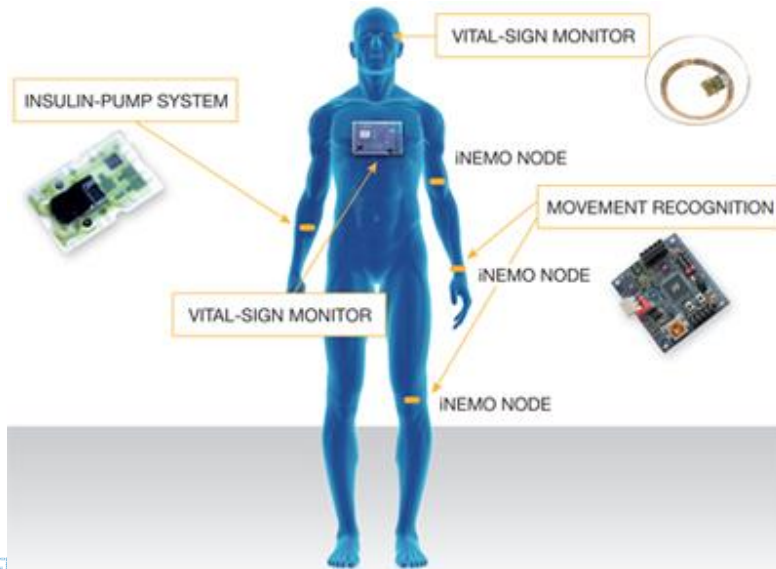


The AutoSar architecture covers many of the protocols necessary for automotive infotainment.

# Content

- About Embedded Systems
- Why Multitasking
- OSEK and AUTOSAR

# Nowerdays, we are surrounded by Embedded Systems....



## But how did it all start?

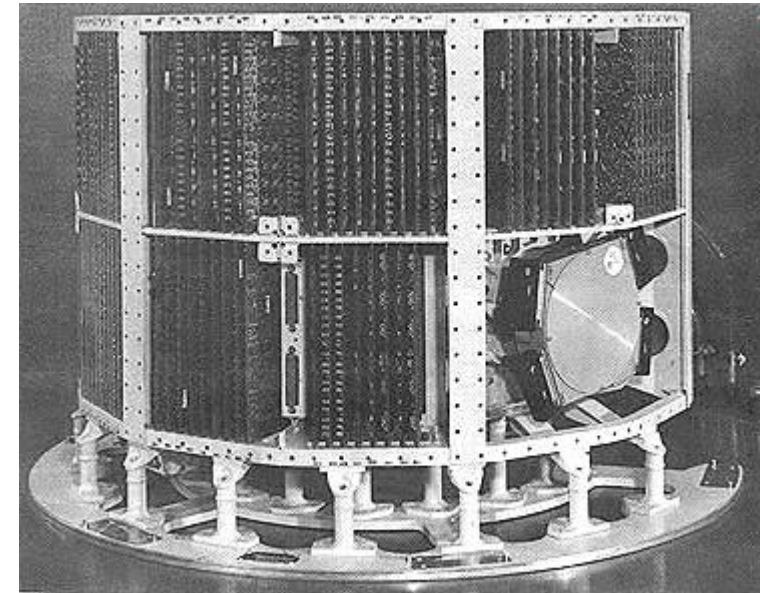
### Back to the 60ies. Apollo Guidance Computer

At the project's inception, the Apollo guidance computer was considered the riskiest item in the Apollo project as it employed the then newly developed monolithic integrated circuits to reduce the size and weight.





# Minuteman Guidance Computer

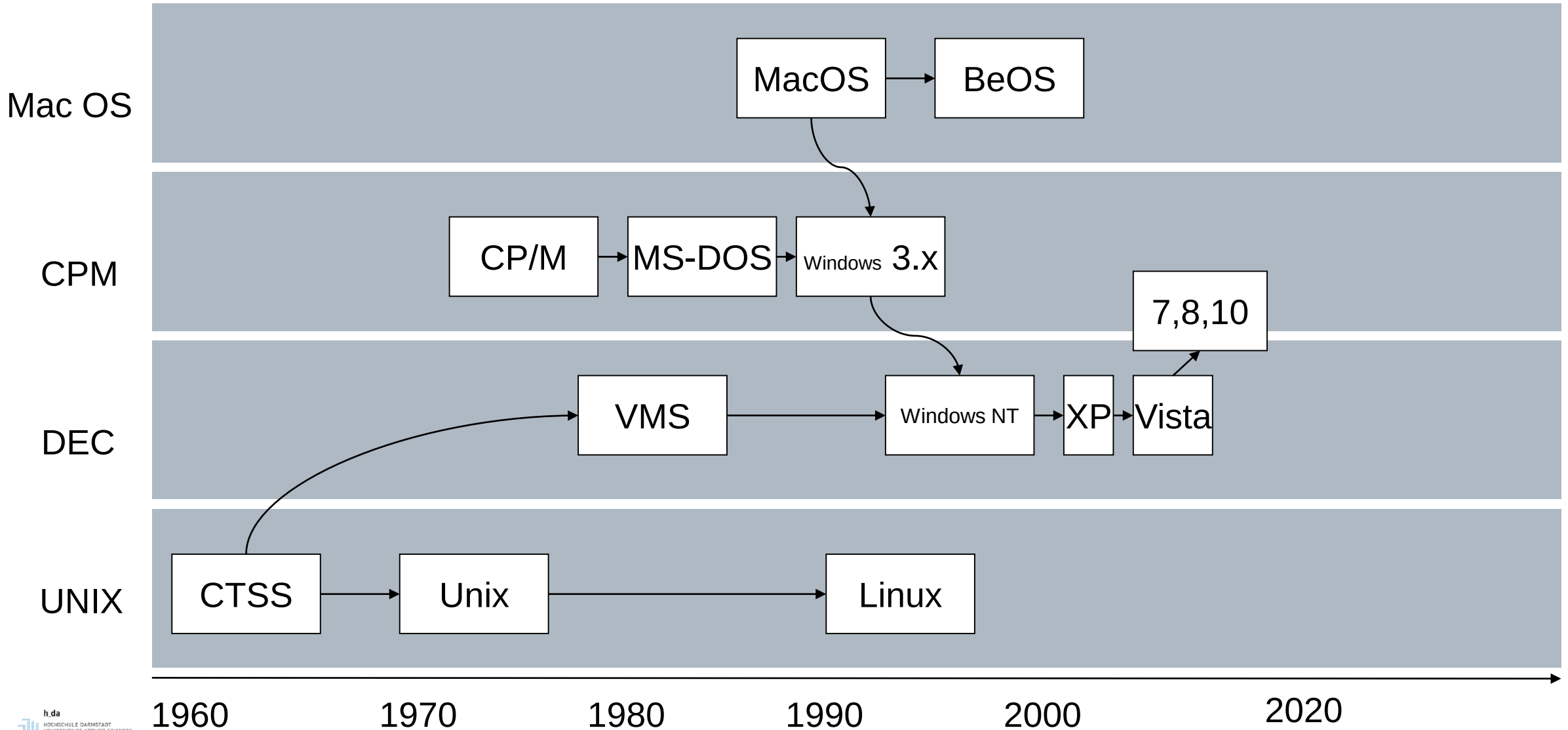


## History of Embedded Systems – 70ies

- One of the first “real” microcontroller was the Intel 8048, released in 1976.
- The 8048 has a Modified Harvard architecture, with internal or external program ROM and 64–256 bytes of internal (on-chip) RAM. The I/O is mapped into its own address space, separate from programs and data.
- It was used in the Magnavox Odyssey video game console, the Roland Jupiter-4 and Roland ProMars analog synthesizers, and (in its 8042 variant) in the original IBM PC keyboard.



# History of mainframe OS



## Early years (1960 – 80)

### Main motivation

- Make best use of the (expensive) hardware

### OS examples

- CTSS - Compatible Time-Sharing System
- UNIX – first cross platform OS

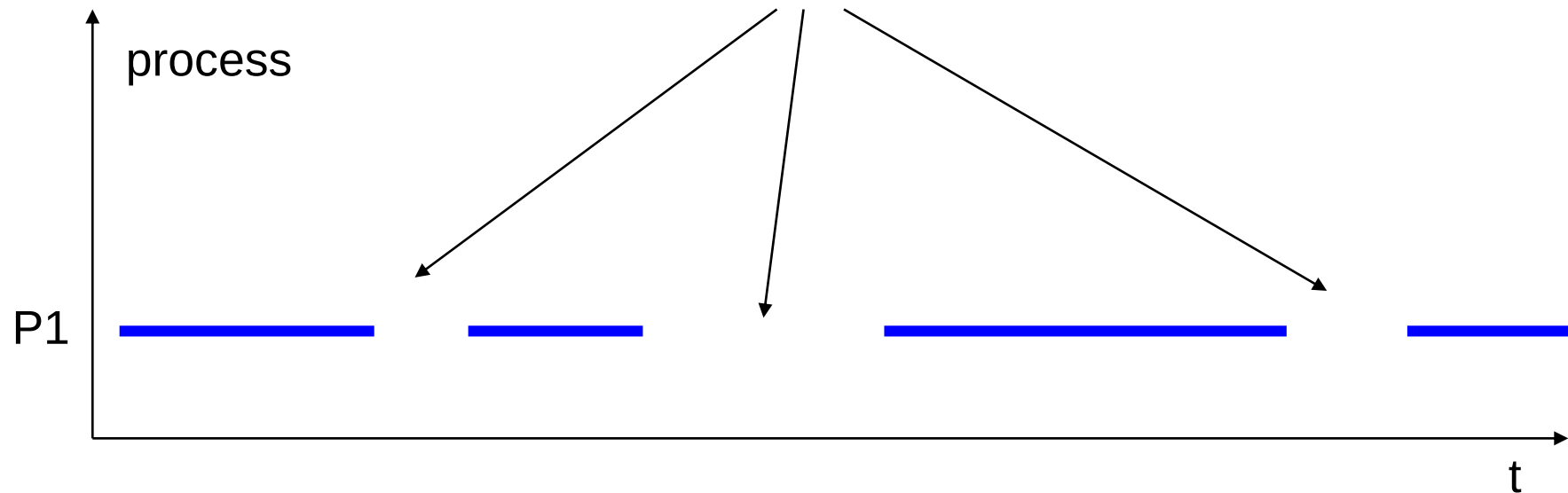




# Single Tasking on a Computer

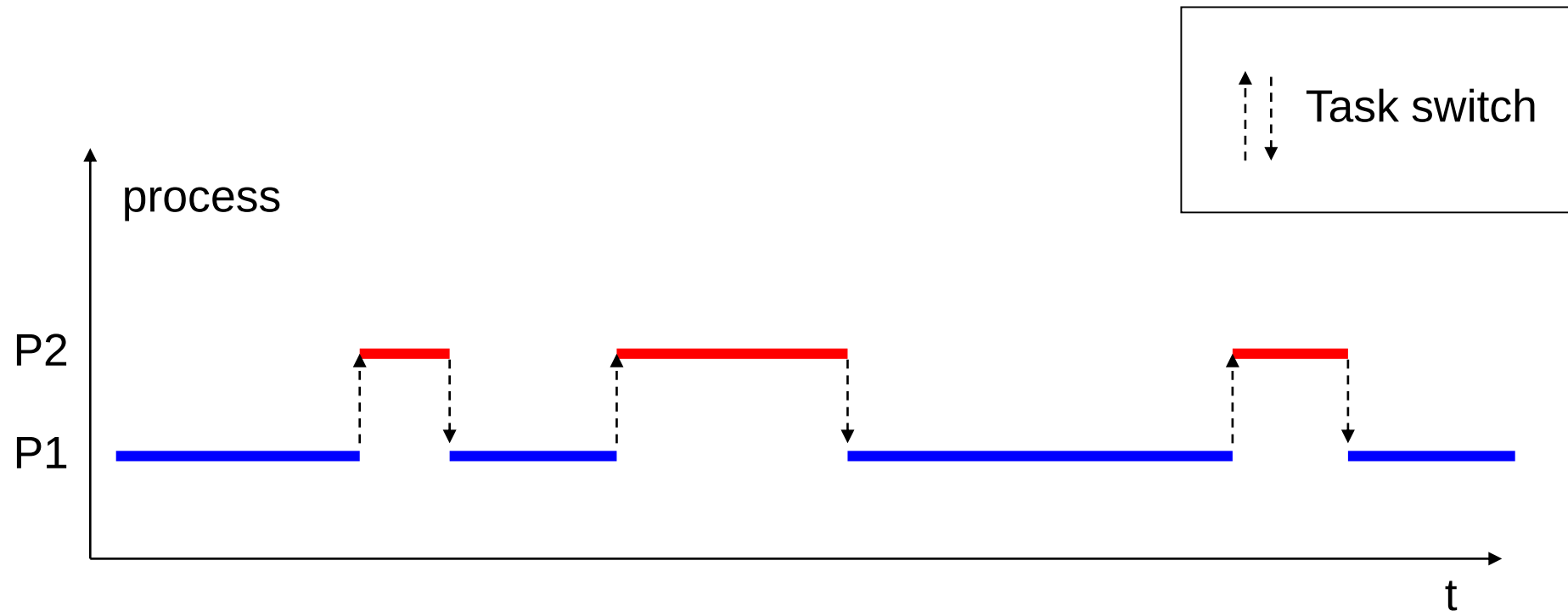
- Execute one program after the other
- Traditional way of computing

Waiting e.g. for resources = Processor Idle Time



# Cooperative Multitasking

- Idea: Use the processor idle time for an additional process
- Also referred to as non-preemptive or time sharing multi tasking



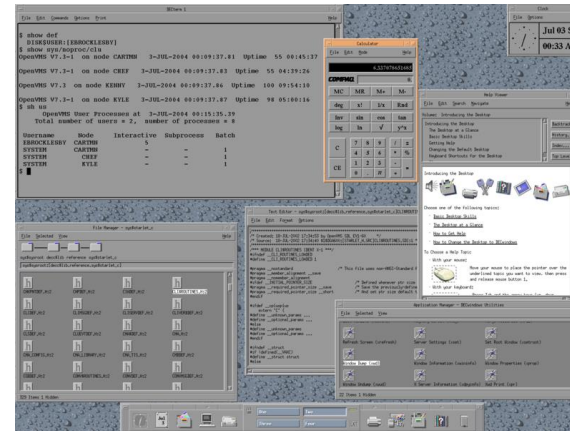
# First office and home computers (1980)

## Main motivation

- Provision of key services
  - Disk Operating Systems
  - Hardware access through BIOS

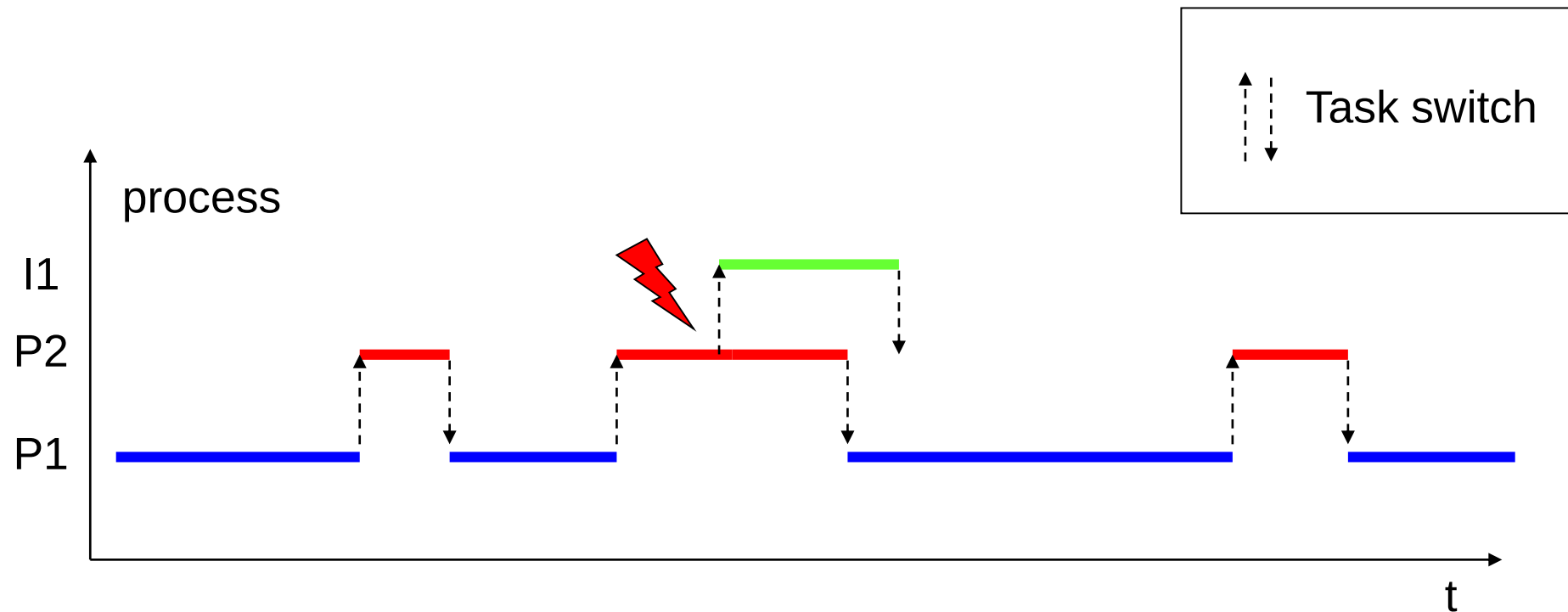
## Examples

- CP/M (1976) – command prompt
- VMS (1978) – Graphical UI
- MS-DOS (1981) – Hardware abstraction
- Windows 3.1
- Amiga OS – First preemptive OS



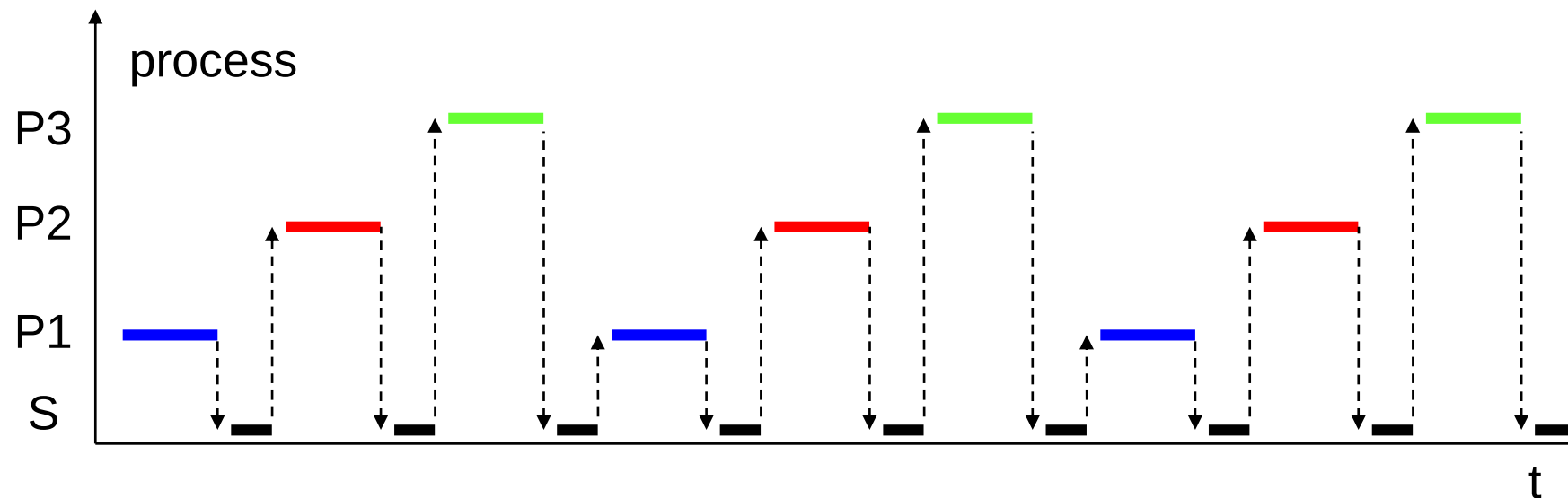
# Preemptive Multitasking

- Idea: Allow interruption of the program execution in case an event with a higher priority occurs



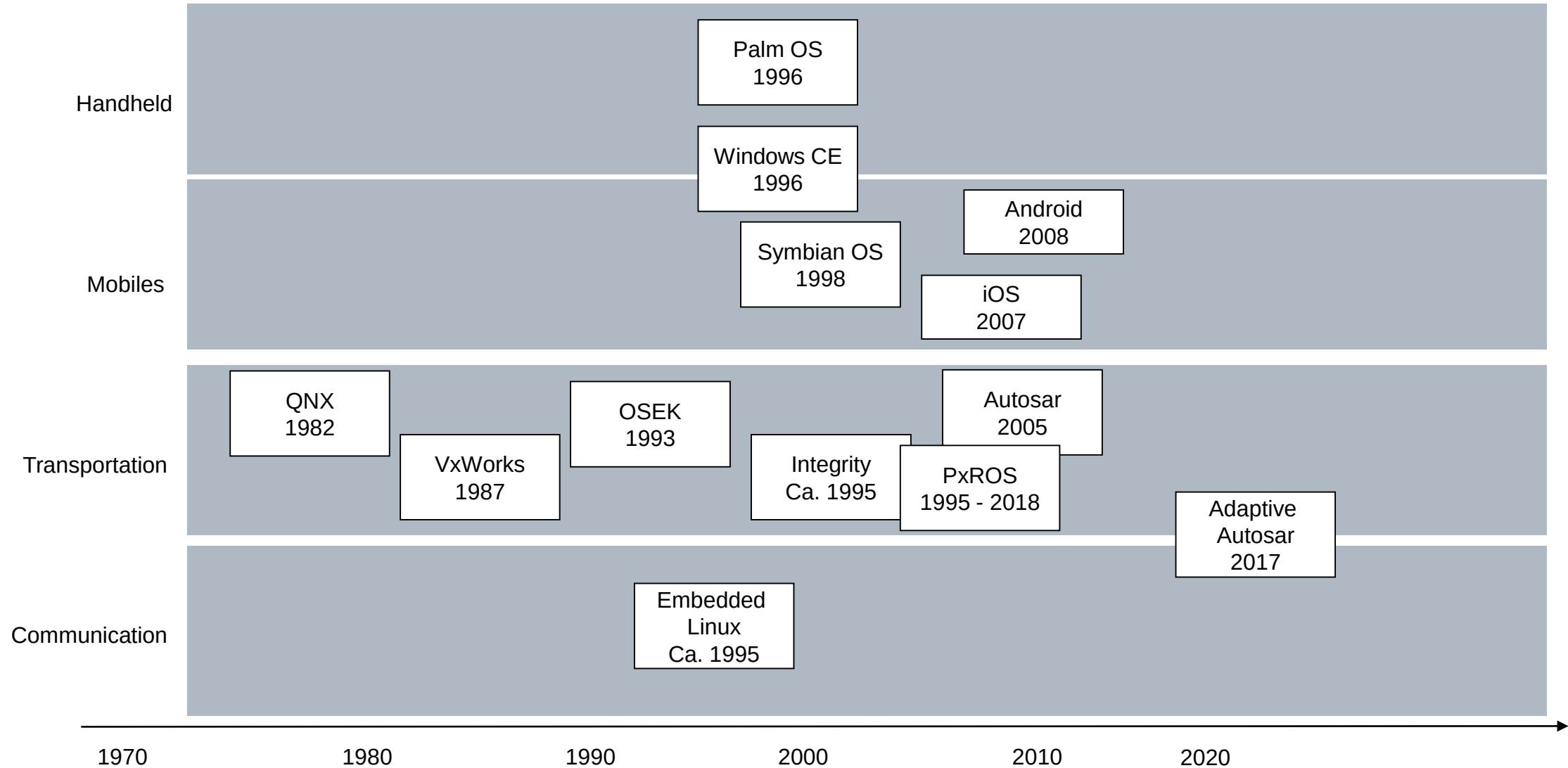
# Scheduled Multitasking

- Idea: Use a constant timeslot for every process
- A timer acts as a high priority interrupt, passing control to the scheduler
- The scheduler decides who gets the CPU time based on its scheduling algorithm
- The time occupied by the scheduler is also referred to as taskswitch or latency time





# History of embedded operation systems



# QNX

- Gordon Bell and Dan Dodge, students at the University of Waterloo in 1980, constructed a first basic real-time kernel and started Quantum Software Systems. In 1982 the first version of QNX was released for the Intel 8088 CPU.
- In 2001 a Posix compliant version, QNX Neutrino, was released.
- In 2004, the company was sold to Harmaan International
- Features
  - As a microkernel-based OS, QNX is based on the idea of running most of the OS in the form of a number of small tasks, known as servers.
  - This approach supports a flexible configuration by simply turning off the servers not needed.
- Application areas:
  - Automotive telematics
  - Power plants
  - Education

# QNX

- Gordon Bell and Dan Dodge, students at the University of Waterloo in 1980, constructed a first basic real-time kernel and started Quantum Software Systems. In 1982 the first version of QNX was released for the Intel 8088 CPU.
- In 2001 a Posix compliant version, QNX Neutrino, was released.
- In 2004, the company was sold to Harmaan International
- Features
  - As a microkernel-based OS, QNX is based on the idea of running most of the OS in the form of a number of small tasks, known as servers.
  - This approach supports a flexible configuration by simply turning off the servers not needed.
- Application areas:
  - Automotive telematics
  - Power plants
  - Education

# VxWorks

- VxWorks is a proprietary, real-time, operating system developed by Wind River Systems of Alameda, California, USA in 1987.
- VxWorks has been ported to a number of platforms including the x86 family, MIPS, PowerPC, Freescale ColdFire, SH-4 and the closely related family of ARM, StrongARM and xScale CPUs.
- Features:
  - multitasking kernel with preemptive and round-robin scheduling and fast interrupt response
  - memory protection to isolate user applications from the kernel
  - Tornado development environment
- Applications
  - Aircraft and military, e.g. Boeing 747 and 787, Apache helicopter
  - Automotive navigation, VDO Dayton
  - Mars Reconnaissance Orbiter, Phoenix Mars Lander
  - Industrial robots e.g. Kuka

# Palm OS

- Palm OS (also known as Garnet OS) is an embedded operating system initially developed by U.S. Robotics' owned Palm Computing, Inc. for personal digital assistants (PDAs) in 1996.
- Support for Freescale and ARM microcontrollers.
- Features
  - Simple, single-tasking environment to allow launching of full screen applications with a basic, common GUI set
  - Handwriting recognition input system called Graffiti 2
  - TCP/IP network access, Serial port/USB, Infrared, Bluetooth and Wi-Fi connections
  - Defined standard data format for personal information
- Application
  - PDA's
  - Smartphones
  - Handheld gaming consoles

# Symbian OS

- On 24 June 1998, Symbian Ltd. was formed as a partnership between Ericsson, Nokia, Motorola and Psion, to exploit the convergence between PDAs and mobile phones.
- Symbian OS is an open operating system, designed for mobile devices, and runs exclusively on ARM processors.
- Symbian OS is the leading OS in the "smart mobile device" market: 67% followed by Microsoft having 13% through Windows CE.
- Features
  - Large networking and communication subsystem
  - UI Framework Layer for third party products
  - Java ME application service layer
- Application
  - Mobiles



# Windows CE

- Windows CE is a variation of Microsoft's Windows operating system for minimalistic computers and embedded systems.
- It is supported on Intel x86 and compatibles, MIPS, ARM, and Hitachi SuperH processors.
- Features
  - Windows compatibility
- Application
  - Handhelds
  - Mobile phones
  - Game consoles

# Embedded Linux / Android

- Embedded Linux is the use of a Linux operating system in an embedded computer system. According to a VDC survey, Linux was used by 18% of embedded engineers.
- Features
  - Linux compatibility
  - Open source
- Applications
  - mobile phones
  - personal digital assistants
  - media players
  - networking equipment
  - machine control and industrial automation
  - navigation equipment and medical instruments

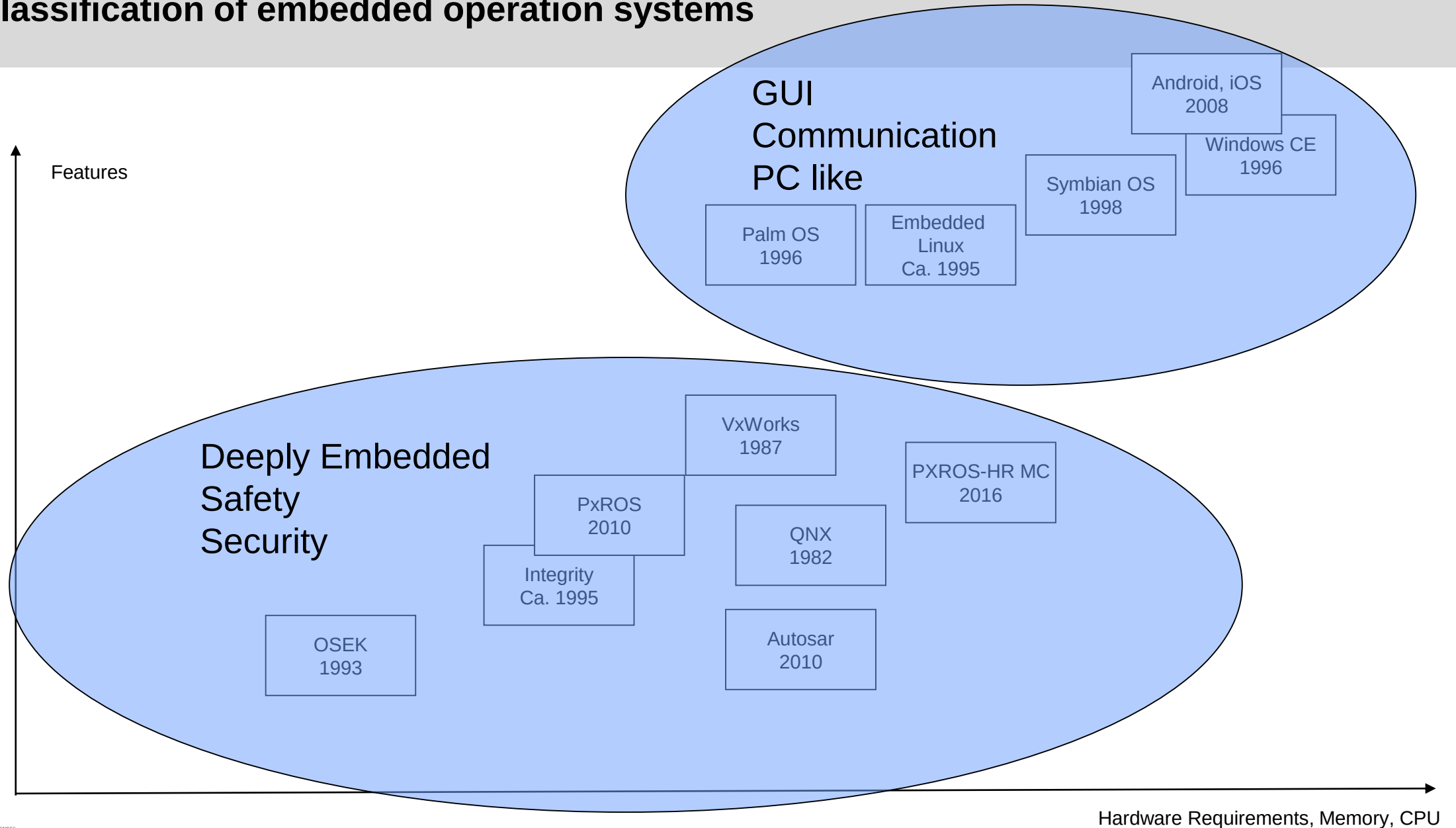
# Integrity / Velocity

- Integrity is a real-time operating system (RTOS) produced and marketed by Green Hills Software. It is royalty-free, POSIX-certified, and intended for use in embedded systems needing reliability, availability, and fault tolerance.
- Supported platforms include variants of ARM and XScale, Blackfin, Freescale ColdFire, MIPS, PowerPC, and x86 computer architectures.
- Features
  - Hardware memory protection
  - Virtual file systems
  - Multi Toolchain
- Applications
  - Military aircraft
  - Automotive
  - Networking, routers

- OSEK (Offene Systeme und deren Schnittstellen für die Elektronik in Kraftfahrzeugen; English: "Open Systems and their Interfaces for the Electronics in Motor Vehicles") is a standards body founded in 1993 by a German automotive company consortium (BMW, Robert Bosch GmbH, DaimlerChrysler, Opel, Siemens, and Volkswagen Group) and the University of Karlsruhe. In 1994, the French cars manufacturers Renault and PSA Peugeot Citroën joined the consortium.
- Implementations provided e.g. by Vector Informatik, 3Soft
- Features
  - Static embedded operating system for 16 and 32bit controllers
  - Specifications include the operating system kernel, the communications stack, and a network management protocol
- Application
  - Automotive powertrain, infotainment, car body

- PXROS-HR is an object oriented real-time operating-system (RTOS) with a <marketing> very modern micro-kernel and outstanding features, especially suited for deployment on advanced multi-core MCUs </marketing>. The latest version of PXROS-HR improves the concepts of encapsulation and robustness by using fine granular hardware protection mechanisms (MPU), available in modern micro-controllers like the AURIX.
- The PXROS-HR operating-system for TriCore has been officially safety approved. HighTec received the certificate confirming the fitness of PXROS-HR for safety-related applications up to SIL2 (IEC61508).

# Classification of embedded operation systems



# General Architecture of a SW-System

An Operating System is a software component of a computer system that is responsible for the management and coordination of activities and the sharing of the resources of the computer



Discussion: What activities and resources are we talking about?

# Experiment 1



## Requirements

- Perform a cyclic measurement
- Print the measured result on the screen
- Start stop the measurement when the corresponding button is pressed

How to translate this into code?

Which problems do we encounter?



## Experiment 2



### Requirements

- Press a button to connect to an external device (via some network)
- Once the device answers, the connection is established
- The established link is shown in the GUI

How to translate this into code?

Which problems do we encounter?

# Example of a pre-emptive multitasking application

- Reactive system
- Parallel execution of more than one independent task



# Requirements for an embedded RTOS

- We need independent implementation units in terms of computation time, called tasks or threads
- The operating system will activate those tasks (using a specific scheduling strategy)
- Tasks can be cyclic (e.g. executed every xyz ms) or event driven (activated by an event). To simplify things – also cyclic events do exist.
- Tasks can be pre-empted by tasks of a higher priority.
- Tasks can manage additional resources like memory, supporting the construction of partitions which cannot interfere with other partitions

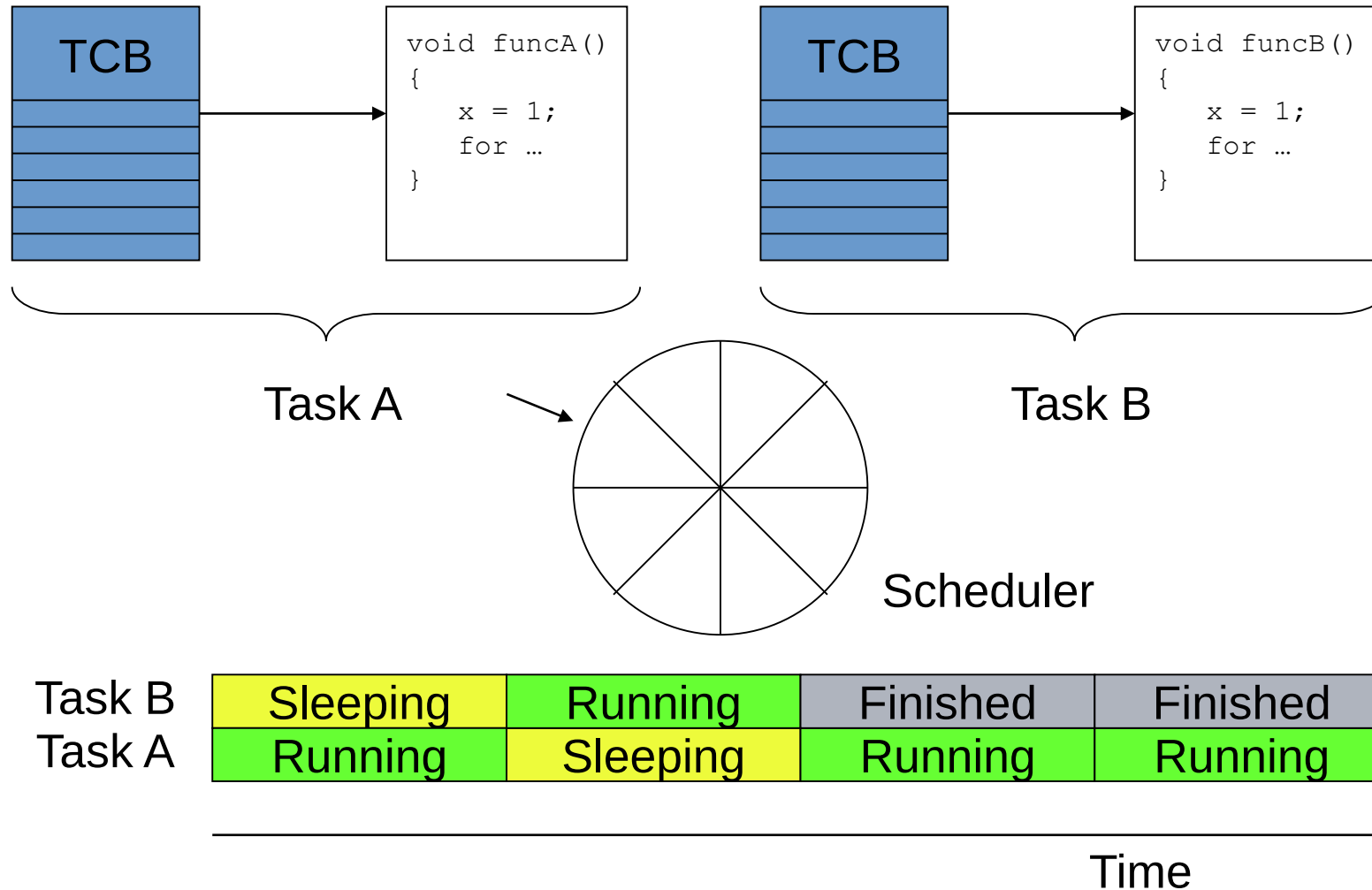
# Definition

A **task** is a **function** called by the **scheduler**.

In addition to a function, a task

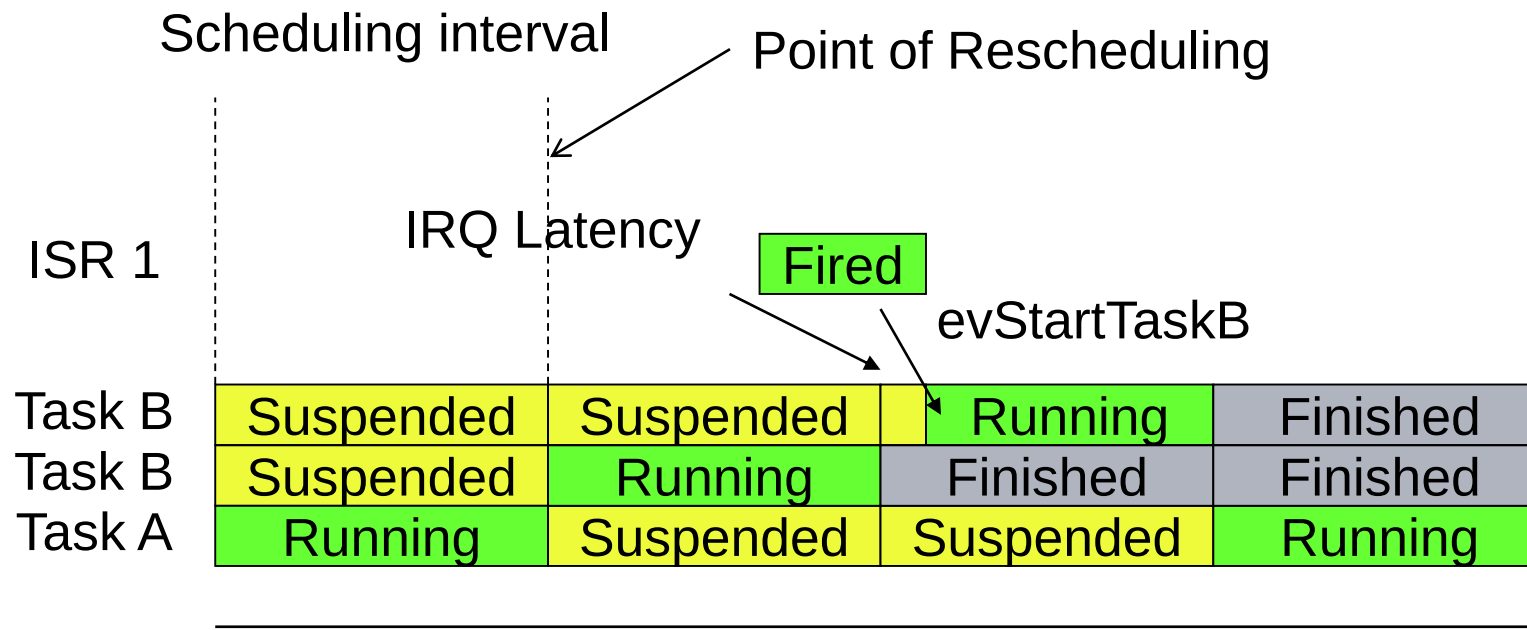
- Has additional meta data like priority, estimated runtime,...(task control block, TCB)
- Has a state (e.g. start, running, waiting,...)
- Can be synchronised with other tasks using OS resources like events, alarms, messages
- Are typically executed in a multitasking context (preemptive or non-preemptive)
- May exist in more than one instance (reentrance)

# Task, Function, Scheduler



# Task State-Time Diagram

- Used to describe the state of tasks over time
- Used to describe message flow between tasks
- Note: In a single controller system only 1 task can have the status running in the same timeslot



## Example Code 1

```
//Init Task - sets up the application tasks
TASK(tsk_init)
{

    //Init MCAL Drivers
    LED_Init();
    SEVEN_Init();

    EE_system_init();

    //Start the alarm with 100ms cycle time
    SetRelAlarm(alarm_ledBlink,100,100);

    ActivateTask(tsk_sevenSet);
    ActivateTask(tsk_background);

    TerminateTask();
}
```

```
//Cyclic Task
TASK(tsk_ledBlink)
{

    static uint8_t counter = 0;

    //After 10 calls == 1s fire the 1s event to
    the sevenSet task
    counter++;
    if (10 == counter)
    {
        counter = 0;
        SetEvent(tsk_sevenSet, ev_1s);
    }

    LED_Toggle(LED_ALL);
    TerminateTask();
}
```

## Example Code 2

```
//Event driven task
TASK(tsk_sevenSet)
{
    static uint8_t count = 0;

    EventMaskType ev = 0;
    while(1)
    {
        WaitEvent(ev_1s | ev_reset);
        GetEvent(tsk_sevenSet, &ev);
        ClearEvent(ev);

        if (ev & ev_1s){
            SEVEN_SetHex(++count);
        }

        if (ev & ev_reset){
            count = 0;
            SEVEN_SetHex(count);
        }
    }
}
```

```
//Background Task
TASK(tsk_background)
{
    while(1)
    {
        //do something with low priority
        __asm("nop");
    }
}
```



# Wrap-Up

- OS History
- Embedded History
- Motivation Multitasking
- First Code Snippets

# Any questions?

